2026

TEST I (2026)

PART I: Introduction to software engineering

TEAM number: 5

Team members (names):

1. Alket Sula → TOPIC 1 (FIRST HALF)
2. Marsila Bello → TOPIC 2
3. Kristina Hyka → TOPIC 3
4. Klevis Rexha → TOPIC 1 (SECOND HALF)
5. Nexhmie Cemeta → TOPIC 4

Writer of the answer to TEST I:

For this test you have to build teams. One answer per team is required.

Remark: Most questions of Test I are directly taken from the question part of the respective topic, in particular from multiple-choice questions. This will also be the principle of examinations: Multiple-choice questions from the topics contribute to 50% of the exam questions.

Send your answer to: Klaus Bothe klaus.bothe@hu-berlin.de and Elinda Kajo Mece ekajo@fti.edu.al

Deadline: 23rd March 2026

Give short answers:
For the multiple-choice questions below, *indicate which is true or false (YES/NO)* ***and*** *explain your choice* ***by reference to a slide*** *(like in sample answer to 1.1 a)* ***and*** *with some* ***very short explanation****.*

Topic 1

1.1 Subfields of software engineering can be found in
a) SWEBOK YES NO
b) Textbooks on SE YES NO
c) Conferences like ICSE YES NO

Sample answer 1.1 a): YES **X** NO
Reason: Topic 1 Slides 91 – 95:
SWEBOK is the official definition of subfields of SE in a systematic way (by a classification)

1.2 Which of the sources mentioned in 1.1. do you prefer?
Explain why (very short answer)

1.2)
Reason: Topic 1, slides 91-95:
I would choose the SWEBOK because it is official definition and has some standart classification for the field (IEEE and SEI Slide 91). Other approaches give some complex definitions and not very clear for the student/individual.

1.3 Which subjects are subfields of software engineering?

a) Project management	YES	NO
b) Programming	YES	NO
c) Data bases	YES	NO
d) Complexity theory	YES	NO
e) Software metrics	YES	NO
f) Cost estimation	YES	NO
g) Development tools	YES	NO
h) Compiler construction	YES	NO
i) Agile software development	YES	NO
j) Refactoring	YES	NO

1.3 a) YES **X** NO
1.3 b) YES **X** NO
1.3 c) YES NO **X**
1.3 d) YES NO **X**
1.3 e) YES **X** NO
1.3 f) YES **X** NO
1.3 g) YES **X** NO
1.3 h) YES NO **X**
1.3 i) YES **X** NO
1.3 j) YES **X** NO

Reason: Topic 1, Slides 11/12/17

All the above subfields are given shortly in the slides as indicated 11 /12/ 17 and some other subfields are shown there which I view as important such as: Reuse, Requiremenets Engineering (very important in the first stage + software architecture), User cases/ User documentation is another important POV to the Subfields that SE includes. These are hardcore concepts in building large scale, reusable code/methods and great function in SE.

1.4 What is the fundamental subject of software engineering (a – g below)?

a) Economical development of high-quality software	YES	NO
b) Development of high-quality programming languages	YES	NO
c) Programming to have an excellent product	YES	NO
d) Complexity theory to develop efficient algorithms	YES	NO
e) Systematic, disciplined, quantifiable approach to the development of software	YES	NO
f) Management of complex data bases	YES	NO
g) Problems of development of complex (large-scale) software	YES	NO

1.4 a) YES **X** NO

Reason: Topic 1, Slide 24

Key focus of SE definition (building scalable software, maintainable, functional, and adaptable)

1.4 b) YES NO **X**

This is a sub dicipline not directly connected to the SE, but iI does effect in planning the software itself.

1.4 c) YES NO **X**

Programming itself is important to SE but it is not its fundamental subject, this is something as programming languages, not depended by SE approach, but simply entering the field as a compulsatory setting.

1.4 d) YES NO **X**

Because this is something that belongs to theoritical computer science.

1.4 e) YES **X** NO

Reason: Topic 1, Slide 56:

Given as the IEEE definition

1.4 f) YES NO **X**

This is known as a subfield of Computer Science and not directly connected to SE.

1.4 g) YES **X** NO

Reason: Topic 1, Slide 24

Sommervile's definition.

1.5 Assess the following statements

a) The field of software engineering originated in the moment when first high-level programming languages were developed, i.e. in the 1950th.

YES NO

b) Software engineering as a discipline was developed as an initiative of NATO.

YES NO

c) Expenses of maintenance are lower than or equal to the development of software.

YES NO

d) It is assumed that expenses of coding are about 20 % of the whole software development process. This percentage (20 %) is about the same for small, medium, and large software.

YES NO

1.5 a) YES NO **X**

Reason: Topic 1, Slides 51-54:

Software engineering became structured later, as shown by SWEBOK defining the field.

1.5 b) YES **X** NO

Reason: Topic 1, Slide 51:

The discipline was formalized early and later structured into fields like in SWEBOK. 1968-1969

1.5 c) YES NO **X**

Reason: Topic 1, Slide 51:

SWEBOK shows Software Maintenance as a major area, implying high cost. Maintenance cost would increase too much in respect with the costs of development.

1.5 d) YES **X** NO

Reason: Topic 1, Slide 73-74:

SWEBOK divides work into many areas, so coding is only a small part. The larger the scale of the project the lower the effort given when taken into consideration programming. Lower scale

projects impact and are based a lot in programming since it is crucial, but when passing to the large scale : architecture, and infrastracture is the issue (cloud systems).

Topic 2

2.1 External vs. internal criteria

External quality criteria of software can be observed by the user of the system. Which of the following criteria are external ones?

a) Robustness YES NO

b) Maintainability YES NO

c) Scalability YES NO

d) Efficiency YES NO

2.1 a) YES **X** NO

2.1 b) YES NO **X**

2.1 c) YES **X** NO

2.1 d) YES **X** NO

Reason: Topic 2, slide 18, 20

In slide 18, *Robustness, Scalability, Efficiency* are listed in the External criteria column. They are defined as "From outside (during the execution) observable properties: what the user can observe".

Maintainability is positioned in the Internal criteria column. They are defined as "Only during the work with the implementation (source-code): only computer specialist can observe".

In slide 20, *Robustness* and *Efficiency* are presented as external quality criteria, while *Maintainability* is presented as internal quality criteria.

2.2 Portability

a) Software is portable, if the effort for the transfer into another environment is significantly smaller than the according redevelopment. YES NO

b) Software is only portable if it can be transferred into a new environment (hardware, operating system) without any changes. YES NO

2.2 a) YES **X** NO

Reason: Topic 2, slide 10

Software is portable, if the change efforts for the transfer are essentially smaller than for the new implementation.

2.2 b) YES NO **X**

Reason: Topic 2, slide 10

Portability does not require to have the software running in other requirements without any modification.

2.3 Scalability

a) Good scalability for a program means to depend on the input (small or big) only in a linear way, i.e. depends in respond time at most linearly on the size of the input.

YES NO

b) Good scalability for a program means to behave for big input like for small input (e.g. same respond time)

YES NO

2.3 a) YES **X** NO

Reason: Topic 2, slide 12

Good Scalability: Efficiency (run time, space) only linear proportional to size of load (e.g. not exponential)

2.3 b): YES NO **X**

Reason: Topic 2, slide 12

Large input should be handled without loss of (linear) efficiency (i.e. only linear increasing time/space)

Topic 3

3.1

What are the fundamental features of the ***V-Model***?

a) In the V-model, test activities are part of the whole development process.

YES NO

b) Test cases are derived from different kinds of software documents, e.g. requirements specification, design, program units. YES NO

c) In the V-model, testing the whole system (acceptance testing) can be completed already at the beginning of the development since acceptance test data are derived from the requirements at the beginning.

YES NO

d) The V-model has been created for large software systems to be able to test in abstraction levels.

YES NO

3.1 a): YES **X** NO

Reason: Topic 3 Slide 67:

Testing activities are included in the whole development process.

3.1 b): YES **X** NO

Reason: Topic 3 Slide 67:

Test cases are derived from requirements, architecture, design, and units.

3.1 c): YES NO **X**

Reason: Topic 3 Slides 68-69:

Test cases are derived early, but testing is executed later so, acceptance testing cannot be done at the beginning.

3.1 d): YES **X** NO

Reason: Topic 3 Slides 46&69:

V-model supports layered testing so testing is done on different abstraction levels (for large systems).

3.2 Which of the three orders of activities are possible according to the V-model:

Order 1 YES NO

Order 2 YES NO

Order 3 YES NO

In addition to YES and NO, *evaluate* each order 1/2/3: best order, possible order, impossible order.

Order 1 (12 single steps):

Requirements definition
Derive: Application scenarios
Architectural design
Derive: Test cases (A)
Detailed design
Derive: Test cases (D)
Unit implementation
Derive: Test cases (U)

Unit testing
Integration testing
System testing
Acceptance testing

Order 2 (12 single steps):
Requirements definition
Architectural design
Detailed design
Unit implementation

Derive: Application scenarios
Derive: Test cases (A)
Derive: Test cases (D)
Derive: Test cases (U)

Unit testing
Integration testing

System testing
Acceptance testing

Order 3 (12 single steps):
Requirements definition
Architectural design
Detailed design
Unit implementation
Derive: Application scenarios
Acceptance testing
Derive: Test cases (A)
System testing
Derive: Test cases (D)
Integration testing
Derive: Test cases (U)
Unit testing

3.2 Order 1 - *possible order:* YES **X** NO
Reason: Topic 3 Slides 70-72:
Activities follow logical sequence that means test cases are derived but not optimally timed.

3.2 Order 2 - *impossible order:* YES NO **X**
Reason: Topic 3 Slide 68:
Test cases must be derived during corresponding phases and here they are derived too late.

3.2 Order 3 - *best order:* YES **X** NO
Reason: Topic 3 Slide 72:
Top-down derivation of test cases, bottom-up testing that matches the idea of the V-model.

3.3 Which of the following statements concerning **agile methods** are true/false?

a) Agile methods in software development are not process models. They are only a collection of useful principles and practices, like close cooperation between customer and software developers, or the frequent delivery of working software.

YES NO

b) Examples of agile methods are Scrum and the V-model.

YES NO

c) A weak point of agile methods is often a bad software architecture

YES NO

d) Agile Methods have been introduced to handle large-scale (complex) software systems

YES NO

3.3 a): YES **X** NO
Reason: Topic 3 Slide 85:
Agile is a collection of practices, not a process model and it consists of principles and practices.

3.3 b): YES NO **X**
Reason: Topic 3 Slide 84:
Scrum is agile, V-model is not (is a process model).

3.3 c): YES **X** NO

Reason: Topic 3 Slide 91:

Weak point: architecture, agile may lead to weak architecture.

3.3 d): YES NO **X**

Reason: Topic 3 Slide 85:

Agile is not specifically designed for large-scale systems, it focuses on flexibility and practices.

Topic 4

4.1 What is true/false about basic concepts in software development?

a) A basic concept is an elementary formalized description mechanism for software documents.

YES NO

b) Basic concepts are necessary to overcome the difficulties of textual descriptions (e.g. misunderstandings, natural language not unique). YES NO

c) Thus, basic concepts complement textual descriptions, i.e. both approaches might be necessary in the same software project. YES NO

d) UML is a basic concept since it collects a lot of different graphical representations which are strongly formalized with respect to the syntax (of that graphical notation).

YES NO

4.1 a) YES **X** NO

Reason: Topic 4, Slide 4

In slide 4, basic concepts are defined as **elementary** and formalized description mechanisms for software documents

4.1 b) YES **X** NO

Reason: Topic 4, Slide 4

Basic concepts are used to specify software during the development process by supporting and complementing natural language descriptions which can be ambiguous or not exact.

4.1 c) YES **X** NO

Reason: Topic 4, Slide 4

Slide 4 states that basic concepts **support and complement** natural language descriptions, meaning both are used together in the same project rather than basic concepts replacing natural language entirely.

4.1 d) YES NO **X**

Reason: Topic 4, Slide 17

Slide 17 states that UML is not a basic concept but a comprehensive description language that collects many elementary techniques such as class diagrams, collaboration diagrams, etc., rather than being one itself.

4.2 Classification:

a) The fundamental classification of basic concepts is based on the view on the software, e.g. view of functionality, view of the objects of the software, view of the classes of the software. Thus, the same software can be described by many different views.

YES NO

b) Since each application is connected with some main view, the same software can be described only by basic concepts of this main view.

YES NO

4.2 a) YES **X** NO

Reason: Topic 4, Slide 15 and Slide 17

Slide 17 confirms that the classification covers 3 levels, starting with which view on the software is supported and states that the same software can be viewed from different views.

4.2 b) YES NO **X**

Reason: Topic 4, Slide 31

Slide 31 shows that most application areas (technical-scientific, real-time, administrative, HCI) draw from several different views simultaneously but no application is locked into just one main view.

4.3 Properties of basic concepts

a) Basic concepts are phase-independent which means that each basic concept can always be used in all software development phases (e.g. specification, design, coding). YES NO

b) Application-independence of basic concepts means that there might be some basic concepts usable in different application areas YES **X** NO

4.3 a) YES NO **X**

Reason: Topic 4, Slide 28

Phase-independence means concepts can **span multiple phases** but it does **not** mean every concept works in **all** phases (e.g. in slide 28 it is shown that Petri nets and use case diagrams are used in only one phase)

4.3 b) YES **X** NO

Reason: Topic 4, Slide 31

The application areas table shows the same basic concepts (like class diagrams, data flow diagrams, pseudo code) appearing **across multiple** application types (technical-scientific, real-time, and administrative share several concepts).